Fire VFX Graph System • Point Cache (URP)

Lite Edition

• • •

Support & Feedback

Email: care@skulmovski.studio • Discord: [Join our community]

If you encounter any issues, please contact me via Discord or Email before submitting a review.

• • •

Table of Contents

About the Lite Edition

Fire VFX — Lite Edition is a streamlined free version of **Fire VFX — Wraps Any Mesh · Point Cache.**

It keeps the core Point Cache workflow for surface-conforming fire on custom meshes, while advanced masking, additional secondary effects and extended customization are available in the full edition.

Feature	Lite Edition	Full Edition
Point Cache surface-conforming fire	✅	✅
Custom 3D mesh support	✅	✅
GPU-driven Visual Effect Graph	✅	✅
Fixed Flame Spawn mode	✅	✅
By Percent Flame Spawn mode	✅	✅
Main Flame controls	✅ Basic	✅ Advanced
Smoke layers	✅ 1 layer	✅ 3 layers
Sparks	✅	✅
Embers	❌	✅
Heat Distortion	❌	✅
Burn Masks	❌	✅ 6 modes
Procedural 3D Noise Burn Mask	❌	✅
UV Texture Burn Mask	❌	✅
Vertex Color Burn Mask	❌	✅
Noise blending with UV / Vertex Color masks	❌	✅
Advanced Smoke 3D Noise Force	❌	✅
Directional / Wind Force	❌	✅
Advanced secondary-layer Spawn Chances	❌	✅
Included Flame Flipbooks	✅ 1	✅ 4
Custom Flame Flipbook assignment	❌	✅
Flame Color Over Life	✅	✅
Fixed Flame Color mode	❌	✅
Per-particle Flame Color Variation	❌	✅
Soft Particle blending	✅	✅
Flame visibility toggle	✅	✅
Full Smoldering setup	❌	✅

Setup Assistant	✅	✅
Exposed parameters	✅ 37	✅ 120+

↑ Table of Contents ↑

⚠️ Critical: If the Fire Doesn't Appear After Import (Known Unity Bug)

Due to a known Unity Visual Effect Graph behavior, a freshly imported `.vfx` asset sometimes does not compile correctly the very first time — especially in a project where the Visual Effect Graph package was just installed, or where the effect is being imported for the first time. If this happens, the fire (or the demo scene) may appear completely broken: no particles, no flame, no smoke.

This is not a problem with your project setup — it's how Unity's VFX Graph compiles shader data on first import.

How to fix it (takes 5 seconds):

1. Open the **VFX Setup Assistant** (via the top Unity menu: *Tools > Skulmovski Studio > Fire VFX — Wraps Any Mesh · Point Cache · URP > VFX Setup Assistant*). Upon opening, the assistant checks and shows you if the **Visual Effect Graph** package is installed in your project *(use the* **Refresh State** *button if you just installed it)*.
2. If the graph is installed, navigate to the **Useful Tools** tab in the assistant and click the **Recompile VFX Graphs** button.
3. Wait a moment for the console to confirm. The fire should immediately start rendering and playing — no need to press Play.

↑ Table of Contents ↑

⚠️ Critical: Default Safe Spawn Settings (Read First)

To prevent unexpectedly heavy GPU load when the effect is first applied to a model with a large Point Cache, the main Flame spawn rate is intentionally set very low by default.

In **Fixed** spawn mode, `Flame_SpawnRateFixed` starts at `1`, so only one Point Cache position is selected for flame spawning each second.

To bring the fire to life, increase `Flame_SpawnRateFixed`, or switch `Flame_SpawnRateMode` to **By Percent** and adjust `Flame_SpawnRateByPercent`.

Smoke and Sparks use their own independent spawn controls:

- `SPB_SpawnRate` — controls Spark spawning.
- `SL1_SpawnPerSecond_EXPENSIVE` — controls Smoke spawning.

Increase spawn values gradually, especially when using Point Caches with a large number of points. Higher Flame, Smoke and Spark spawn values can significantly increase the number of simulated particles and GPU cost.

↑ Table of Contents ↑

⚠️ Important: This Effect Is Designed for Stationary Objects

Spawn points are recalculated every frame based on your object's current Transform, so newly spawned particles appear at the object's current position.

However, particles that have already spawned are simulated in world space and are not re-attached to the object afterward.

This means that if you move, rotate, or animate the object while it is burning, previously spawned particles remain at their original world-space positions and finish their lifetime there.

Practical result:

- **Object stays still** → fire remains stable and correctly conforms to the baked surface.
- **Object moves slowly or occasionally** → a light trailing effect may appear.
- **Object moves quickly or continuously** → visible particle trails may remain behind the object.

Best suited for stationary props, buildings, torches, campfires, and other objects intended to remain in place while burning.

↑ Table of Contents ↑

⚠️ IMPORTANT: Optimization for Production

Out of the box, the effect uses settings intended to make initial setup safe and predictable. Before shipping your final game, we recommend checking the following:

Automatic Bounds

By default, the VFX Graph uses Automatic Bounds. This helps prevent the effect from being unexpectedly culled while you are setting it up, but Unity must update the bounds continuously.

Once the fire is configured for a stationary object, record the bounds and switch the relevant VFX systems to **Recorded** or **Manual** bounds where appropriate.

Particle Capacity

Each active particle layer has a fixed Capacity inside the VFX Graph:

- Main Flame
- Sparks
- Smoke Layer 1

If Spawn Rate and Lifetime values are pushed high enough to reach a layer's Capacity, older particles may disappear early or new particles may fail to spawn.

If a specific effect legitimately needs a higher particle count, open the `.vfx` graph and increase the Capacity for that layer manually.

Always increase spawn values gradually and monitor GPU performance, especially on large Point Caches.

↑ Table of Contents ↑

Project Requirements

For this asset to work correctly, your project must meet the following requirements:

- **Visual Effect Graph** — make sure the Visual Effect Graph package is installed through Unity Package Manager.
- **Universal Render Pipeline (URP)** — the Lite edition is designed for URP.
- **Depth Texture** — enable Depth Texture in your active URP Asset. It is required for the Soft Particle fading used by the Flame, Smoke, and Sparks.
- **ZWrite on scene materials** — opaque scene materials should write to the depth buffer so Soft Particles can fade correctly when intersecting geometry.

↑ Table of Contents ↑

Using Setup Assistant

Before manually configuring your project or opening the Demo Scene, we highly recommend using the built-in **Setup Assistant** to check your project and scene configuration.

How to open: Go to the top Unity menu and select Tools > Skulmovski Studio > Fire VFX — Wraps Any Mesh · Point Cache · URP > VFX Setup Assistant.

⚠️ IMPORTANT: The Assistant is a Mentor, Not a "Magic Wand"

The Setup Assistant is shared with the full edition and may also display checks used by features that are not included in the Lite Edition. For Lite, the essential requirements are listed in the **Project Requirements** section above.

Project-Wide Settings

These settings include Render Pipeline configuration, Visual Effect Graph package checks, and other project-level setup validation.

Scene Settings

This section evaluates settings exclusively for the currently open scene. This is also critical for correct rendering and preventing visual artifacts.

💡 **The Goal of Categories 1 & 2:** They do not check "everything in the world." Their primary objective is to guarantee that you can **SUCCESSFULLY OPEN THE DEMO SCENE** without graphical glitches, hidden bugs, or console errors. Once you have successfully tested the demo scene, you can use the Assistant in your custom scenes to quickly spot missing checkboxes.

Useful Tools

This category is optional. It contains helpful scripts and micro-utilities to simplify your workflow, such as a tool to force-recompile stuck VFX Graphs, and a quick shortcut to open Unity's Point Cache Bake Tool.

Additional Notes

Important technical tips and recommendations regarding optimization, point cache limits, and physics behavior.

↑ Table of Contents ↑

Fast Ignition (Quick Start)

If you are already familiar with VFX Graph, this is the fastest way to ignite your model:

1. Open the **VFX Setup Assistant** and use the **Open Point Cache Bake Tool** shortcut, or go to `Window > Visual Effects > Utilities > Point Cache Bake Tool`.
2. Select your model's **Mesh** and bake its surface positions.
3. Drag your 3D model into the Scene.
4. Add a **Visual Effect** component and assign the Lite Fire VFX asset.
5. Enter the exact baked point count into `PointCacheCount`.
6. Expand the generated `.pCache` file and assign its position map to `PointCachePositionMap`.
7. Increase the Flame Spawn Rate from its safe default value.

Your object should now begin burning.

↑ Table of Contents ↑

How to Ignite Custom Models (Detailed Point Cache Workflow)

Step 1. Open the Point Cache Bake Tool

Go to: `Window > Visual Effects > Utilities > Point Cache Bake Tool`

Step 2. Load the Mesh and Choose the Point Count

- Find your model in the Project window.
- Expand it and drag the **Mesh** itself into the Point Cache Bake Tool.
- Choose an appropriate **Point Count**.

Start with a relatively low value and increase it only if the fire coverage looks too sparse.

Step 3. Bake Settings

- **Distribution:** `Random Uniform Area`
- For the Lite edition, only the baked **position data** is required for the standard fire workflow.
- **File Format:** `Ascii`

Step 4. Save the Point Cache

Click **Save to pCache file...** and save the generated file inside your project.

Step 5. Add the Fire Effect

- Drag your 3D model into the Scene.
- Add a **Visual Effect** component.
- Assign the Lite Fire VFX graph.
- Expand the generated `.pCache` file in the Project window.
- Drag its position map into `PointCachePositionMap`.

Step 6. Synchronize the Point Count

Set `PointCacheCount` to exactly the same number of points used when baking the Point Cache.

If the values do not match, the effect will not spawn correctly across the available baked positions.

Step 7. Increase the Flame Spawn Rate

The default Flame Spawn Rate is intentionally low.

Increase `Flame_SpawnRateFixed`, or switch `Flame_SpawnRateMode` to **By Percent** and adjust `Flame_SpawnRateByPercent`.

Increase the value gradually while checking both visual density and performance.

↑ Table of Contents ↑

Choosing the Right Point Count

There is no universal Point Count value. The right number depends on the size and surface area of your model, as well as the visual density you want.

- **Start low.** Bake a relatively small number of points first and check the result in the Scene view.
- **Judge by visual coverage.** Small objects may need only a few points, while larger or more complex meshes may require hundreds or more to avoid visible gaps.
- **Increase gradually.** If the fire looks too sparse, increase the Point Count and bake again instead of immediately using very large values.
- **Monitor performance.** Higher Point Counts can lead to more active Flame, Smoke, and Spark particles depending on your spawn settings.
- **For large meshes, consider By Percent mode.** It scales Flame spawning relative to `PointCacheCount` and is often easier to manage when using large Point Caches.

The goal is to achieve the visual density you need with the lowest practical Point Count.

↑ Table of Contents ↑

Spawn Settings & Layer Optimization

The Lite edition keeps a simplified spawn-control workflow for the Main Flame, Sparks, and a single Smoke layer.

Main Flame Spawn

Flame spawning is controlled by `Flame_SpawnRateMode`.

Fixed
Uses `Flame_SpawnRateFixed` to define how many Point Cache positions are selected for Flame spawning each second.

By Percent
Uses `Flame_SpawnRateByPercent` to spawn Flame relative to the total `PointCacheCount`.

By Percent is useful when working with models that use very different Point Cache sizes, because the spawn amount scales with the baked point count.

Increase Flame spawn values gradually. Very large Point Caches combined with high Spawn Rate and long particle Lifetime can produce a large number of active particles.

Sparks

Sparks use their own independent spawn control: `SPB_SpawnRate`

Increase this value to produce more Sparks. Higher values increase particle count and GPU cost.

Smoke

The Lite edition contains one Smoke layer.

Smoke spawning is controlled by: `SL1_SpawnPerSecond_EXPENSIVE`

Increase this value carefully. Dense Smoke can become significantly more expensive when combined with large fire effects or long particle lifetimes.

General Optimization

For all layers, adjust Spawn Rate together with particle Lifetime and visual density.

More particles do not always produce a better-looking effect. Use the lowest values that provide the required Flame, Smoke, and Spark density for your specific model.

Troubleshooting & Tips

❓ Why Is the Fire Not Visible After Import?

If the Fire VFX does not render correctly immediately after import, open the **VFX Setup Assistant** and use the **Recompile VFX Graphs** tool.

Unity Visual Effect Graph assets can occasionally require recompilation after first import.

❓ Why Is the Fire Very Small or Barely Visible?

The Main Flame uses a very low default Spawn Rate for safety.

Increase `Flame_SpawnRateFixed`, or switch to **By Percent** mode and adjust `Flame_SpawnRateByPercent`.

Also make sure that:

- `PointCacheCount` matches the exact number of baked points;
- `PointCachePositionMap` is assigned correctly;
- the Point Cache was baked from the correct Mesh.

❓ Why Are Particles Invisible or Fading Too Strongly Near Geometry?

Check the corresponding **Soft Particle Fade Distance**.

The Lite edition exposes Soft Particle fading for:

- Flame
- Smoke
- Sparks

If the value is too large for the scale of your object, particles close to geometry may become almost completely transparent.

Reduce the Fade Distance until the intersection looks correct for your scene scale.

❓ Why Does Fire Leave a Trail When the Object Moves?

Already-spawned particles are simulated in world space and do not remain attached to the moving object.

New particles spawn from the object's current Point Cache positions, while older particles finish their lifetime at their previous world-space positions.

The effect is therefore best suited for stationary objects.

❓ Why Are Some Particles Disappearing?

Each particle layer has a fixed Capacity inside the VFX Graph.

If Spawn Rate and Lifetime values create more active particles than the configured Capacity allows, older particles may disappear early or new particles may fail to spawn.

Reduce Spawn Rate or Lifetime, or manually increase the corresponding Capacity inside the `.vfx` graph if your effect genuinely requires more particles.

↑ Table of Contents ↑

Technical Reference: Parameter Glossary

Below is a breakdown of all exposed parameters available in the **Lite Edition** VFX Graph Inspector.

Important!

- **PointCachePositionMap** — the position texture map extracted from your baked `.pCache` file. This map provides the surface positions used for fire spawning.
- **PointCacheCount** — the exact number of points baked into the Point Cache. This value must match the Point Count used when creating the `.pCache`.

Base Toggles

- **FIRE** — master toggle for the entire fire effect.
- **Flame_ShowVisual** — enables or disables the visible Flame particles while keeping the underlying fire system active.
- **Smoke** — enables or disables the Smoke layer.
- **Sparks** — enables or disables the Sparks layer.

(!!!) Adjust Very Carefully! Read Docs First.

- **Flame_SpawnRateMode** — switches the Main Flame spawning between **Fixed** and **By Percent** modes.
- **Flame_SpawnRateFixed** — defines how many Point Cache positions are selected for Flame spawning per second when Fixed mode is used.

- **Flame_SpawnRateByPercent** — defines Flame spawning as a percentage of the total `PointCacheCount` when By Percent mode is used.

01. Flame Base (FLB)

- **FLB_FrameRate** — controls the playback speed of the included Flame flipbook animation.

02. Flame Advanced (FLA)

- **FLA_SpawnSizeRnd** — minimum and maximum randomized initial size of Flame particles.
- **FLA_LifetimeRnd** — minimum and maximum randomized lifetime of Flame particles.
- **FLA_ColorOverLife** — controls Flame color and alpha over the particle lifetime.
- **FLA_BaseWidthInUnit** — base width multiplier for Flame particles.
- **FLA_BaseHeightInUnit** — base height multiplier for Flame particles.
- **FLA_SoftParticleFadeDistance** — controls how gradually Flame particles fade when intersecting scene geometry.
- **FLA_PivotOffset** — offsets the Flame particle pivot relative to its spawn position.

03. Sparks Base (SPB)

- **SPB_SpawnRate** — controls the number of Sparks spawned over time.
- **SPB_LifetimeRnd** — minimum and maximum randomized Spark lifetime.
- **SPB_SizeRnd** — minimum and maximum randomized initial Spark size.
- **SPB_SizeOverLife** — controls Spark size over the particle lifetime.
- **SPB_ColorOverLife** — controls Spark color and alpha over the particle lifetime.
- **SPB_ScaleXY** — scales the Spark particles on the X/Y axes.
- **SPB_SpawnVelocityFrom** — minimum randomized initial velocity used when Sparks are spawned.
- **SPB_SpawnVelocityTo** — maximum randomized initial velocity used when Sparks are spawned.
- **SPB_SoftParticleFadeDistance** — controls how gradually Sparks fade when intersecting scene geometry.

04. Smoke Layer (SL1)

- **SL1_SpawnPerSecond_EXPENSIVE** — controls Smoke spawning. Higher values can become significantly more expensive, especially when combined with long Smoke lifetimes.
- **SL1_SpawnAngleZRnd** — randomized starting Z-axis rotation range for Smoke particles.
- **SL1_LifetimeRnd** — minimum and maximum randomized Smoke lifetime.
- **SL1_SpawnPositionOffset** — offsets the Smoke spawn position relative to the Point Cache fire position.
- **SL1_SizeRnd** — minimum and maximum randomized initial Smoke size.
- **SL1_VelocityRndFrom** — minimum randomized initial Smoke velocity.
- **SL1_VelocityRndTo** — maximum randomized initial Smoke velocity.
- **SL1_AngularVelosityZRnd** — minimum and maximum randomized Z-axis angular velocity for Smoke particles.
- **SL1_ColorOverLife** — controls Smoke color and alpha over the particle lifetime.
- **SL1_SizeOverLife** — controls Smoke size over the particle lifetime.
- **SL1_SoftParticleFadeDistance** — controls how gradually Smoke particles fade when intersecting scene geometry.

↑ Table of Contents ↑